

Card Position Rules on the Canvas

Branch AI / Flowstate — Technical reference

Generated: May 25, 2026

Card positions live in world-space { x, y } and are rendered as CSS left / top on absolutely positioned nodes (components/Card.tsx). The rules split into a core policy, constants, connection topology, and when each code path moves cards.

1. Core policy (absolute positions)

Source: lib/canvasLayout.ts

Rule	Detail
Default	Every card has a fixed absolute <code>(x, y)</code> . Nothing auto-shifts except the cases below.
Vertical chains	Cards linked by bottom !' top connections (same-chat follow-ups) are re-snapped when a parent's height changes so the gap stays canonical.
Lateral branches	Cards linked by left/right plugs stay where placed, except during explicit band repair (see §7).
Artifacts	Canvas artifact nodes are never moved by card resize or vertical relayout.

Production resize path uses full chain relayout (relayoutVerticalChainOf), not a simple delta shift — though shiftBottomAttachedSubtrees still exists for delta-based shifting (tests only today).

2. Layout constants

Constant	Value	Role
<code>FOLLOW_UP_GAP</code>	<code>40</code>	Vertical gap between parent bottom and child top
<code>CARD_WIDTH</code> / <code>BRANCH_CARD_WIDTH</code>	<code>420</code>	Card width; branch X math assumes this
<code>BRANCH_HORIZONTAL_GAP</code>	<code>420</code> (= card width)	Horizontal offset between source and first branch on a side
<code>COLUMN_STEP</code>	<code>840</code>	Column spacing in auto-layout (<code>CARD_WIDTH + BRANCH_HORIZONTAL_GAP</code>)
<code>EMPTY_CARD_HEIGHT</code>	<code>88</code>	Fallback height for empty cards
<code>FALLBACK_CARD_HEIGHT</code>	<code>240</code>	Fallback when no measured size

3. How height is measured (drives Y placement)

getLayoutCardBounds() (lib/canvasMeasure.ts):

1. Prefer live DOM size via [data-canvas-card="..."] when mounted.
2. Else use stored card.size, or fallbacks from getCardBounds().

setCardSize in Card.tsx runs on ResizeObserver when content/layout changes, which triggers vertical chain relay layout.

4. Vertical follow-up chain (bottom connections)

4.1 Canonical follow-up position

```
x = parent.position.x
y = parent.position.y + parentHeight + FOLLOW_UP_GAP    // FOLLOW_UP_GAP = 40
```

Implemented in computeFollowUpPosition().

- X: same as parent.
- Y: parent bottom + 40px gap.

4.2 Which child counts as "the" follow-up

getFollowUpChild() only considers connections where fromSide === "bottom" or fromSide == null.

If multiple bottom children exist, pick:

1. Lowest position.y
2. Tie-break: earlier in cardOrder

4.3 Full chain relay layout

layoutVerticalChain(startParentId) walks the single follow-up chain and assigns each child computeFollowUpPosition, then continues from that child.

findVerticalChainRoot(cardId) walks up bottom connections to the top of the chain before relay layout.

4.4 When this runs in the app

Trigger	Function	Effect
Card resizes (<code>`setCardSize`</code>)	<code>`relayoutVerticalChainOf`</code>	Re-snaps entire bottom-connected chain under that chat
New follow-up (<code>`createFollowUp`</code>)	<code>`computeFollowUpPosition`</code> , then <code>`syncFollowUpChildPosition`</code> (2× rAF)	Ensures DOM-measured parent height is applied
Load snapshot (<code>`hydrateFromSnapshot`</code>)	<code>`repairVerticalChainsOnly`</code>	Fixes stale vertical gaps; **does not** move lateral branches

4.5 Invariant check

`followUpInvariantHolds(parent, child)` is true when the child is exactly at `computeFollowUpPosition`.

5. Lateral branches (left/right connections)

5.1 Y band (`childBandY`)

For a source card, branch Y is:

- The minimum Y among all outgoing children (any side), if any exist; else
- `source.y + sourceHeight + FOLLOW_UP_GAP` (same band as a new follow-up would use)

Used by `createBranch` and `createBranchAt`.

5.2 X placement (`createBranch`)

Slot index = count of existing branches on that side from the same source.

Right:

$$x = \text{source.x} + \text{CARD_WIDTH} + \text{BRANCH_HORIZONTAL_GAP} + (\text{CARD_WIDTH} + \text{BRANCH_HORIZONTAL_GAP}) \times \text{slot}$$

Left:

$$x = \text{source.x} - \text{CARD_WIDTH} - \text{BRANCH_HORIZONTAL_GAP} - (\text{CARD_WIDTH} + \text{BRANCH_HORIZONTAL_GAP}) \times \text{slot}$$

(`lateralStep = BRANCH_CARD_WIDTH + BRANCH_HORIZONTAL_GAP`)

5.3 Drag-to-place branch (`createBranchAt`)

- Y: `childBandY` (not pointer Y).
- X: `pointerWorld.x - BRANCH_CARD_WIDTH / 2` (centers card on drop).

Connection: `fromSide` = dragged side; `toSide` = left if from right, else right.

5.4 Lateral band repair (library; limited production use)

repairLateralBranchBands() aligns each left/right branch (and entire subtree under it via BFS on all connections) so the branch root's Y matches:

parent.y + parentHeight + FOLLOW_UP_GAP

Used inside relayLayoutChildrenOf and repairCanvasLayout — not called on hydrate today (only repairVerticalChainsOnly is). Fuller repair helpers are covered by tests.

6. User drag rules

- Who can drag: Only root cards (parentCardId === null).
- What moves: moveSubtree(rootId, dx, dy) — BFS over all outgoing connections (bottom + left/right + any subtree).
- Coordinate conversion: Screen delta ÷ viewport.scale.
- Threshold: 5px before drag counts as a move.
- Blocked when: Plug drag active, or pointer on plugs / text controls.

Follow-up cards (with parentCardId set) are not individually draggable.

7. Creation & placement entry points

Action	Position rule
`createRootCard` / `createRootCardWithAttachment`	Caller-supplied `(x, y)`
`createFollowUp`	`computeFollowUpPosition` + delayed resync
`createBranch`	Computed X slot + `childBandY`
`createBranchAt`	Centered X from drop + `childBandY`
`autoLayoutCanvas` (context menu)	`computeAutoLayout` — see §8

8. Auto-layout (explicit user action)

computeAutoLayout (lib/autoLayout.ts) — only via Auto layout in the canvas context menu:

1. Find independent roots (no incoming connection), sorted by current position.y.
2. For each root tree:
 - Build vertical column chain via getFollowUpChild (same rules as §4.2).
 - Place chain in column colIndex at x = colIndex × COLUMN_STEP.
 - Preserve each root's current position.y as anchorY; stack children downward with cardHeight + FOLLOW_UP_GAP.
 - Recurse into lateral branch roots (sorted by Y), each in the next column.

Auto-layout overwrites positions for all cards it reaches; it does not run automatically on edit or load.

9. Related non-card positioning (context)

These share world coordinates but are separate from card layout:

- Canvas artifacts: default spawn to the right of source card ($\text{card.x} + \text{cardW} + 24$), with overlap nudge; draggable via `moveCanvasArtifact`.
- Text labels: absolute position; moved via delta on drag.

10. Summary table

Situation	X	Y
Follow-up under parent	$= \text{parent.x}$	$\text{parent.bottom} + 40$
Parent grows/shrinks	Unchanged for parent	Descendants in bottom chain re-snapped
Lateral branch (new)	Side-based column math	<code>`childBandY`</code> band
User drags card	$+dx$ for subtree	$+dy$ for subtree
Hydrate	Unchanged unless in vertical chain	Re-snapped for vertical chains only
Auto-layout	$\text{Column index} \times 840$	Re-stacked from root anchor Y

11. Source files

File	Responsibility
<code>`lib/canvasLayout.ts`</code>	Layout math and repair helpers
<code>`lib/store.ts`</code>	When rules run (create, resize, hydrate, drag)
<code>`lib/autoLayout.ts`</code>	Column auto-layout
<code>`lib/canvasMeasure.ts`</code>	DOM vs stored height for layout
<code>`lib/canvasNodeBounds.ts`</code>	Width/height constants and fallbacks
<code>`components/Card.tsx`</code>	Render position, drag, resize !' relayout

End of document